

Liferay Training

Objectives:

- Establish foundational Liferay understanding
- Learn to match business needs with appropriate Liferay components
- Gain hands-on experience implementing common business solutions

Agenda - Day 1

- Introductions, Schedule and Training Format
- Liferay Components Overview
- Mock University Site Control Panel Demo
- Local Dev Env Installation (Demo/Workshop)
- Web Content (Demo/Workshop)
- StyleBooks, Fragments, Page Templates (Demo/Workshop)
- Q&A / Open Workshop

Agenda - Day 2

- Using the docker image locally
- Custom Portlets (Demo/Workshop)
- Architecture Basics
- Common Configuration tasks (Demo/Workshop)
- Admin tasks(logging, deployments, etc)
- Objects (Demo/Workshop)
- Liferay Resources (slack channel, support desk, etc)
- Q&A / Open Workshop

Agenda - Day 3

- Service Builder (Demo/Workshop)
- Workflows (Demo/Workshop)
- Job Scheduler (Demo/Workshop)
- JSON Web Services (Demo/Workshop)
- Client Extensions (Demo/Workshop)
- Q&A / Open Workshop

Demo – Workshop Application

- Mock University Faculty & Student Portal
- Walkthrough of components
- You will build this yourself on your laptops

Liferay 7.4 Components (1 of 3)

Component	Purpose	Primary Use Cases	Customizable By	Tech Skills Needed	Best For
Fragments	Reusable front-end blocks of HTML/CSS/JS	Page sections, banners, CTAs, reusable layouts	Developers, Designers	Low (HTML/CSS/JS)	Content creators, designers
Widgets (Portlets)	Mini applications that provide functionality or display dynamic content	Blogs, asset listings, custom logic, integrations	Developers	High (Java, JSP, JS frameworks)	Developers
Client Extensions	Injects client-side JS/CSS or config without server changes	Custom JS/CSS, 3rd-party tools, headless UI, analytics	Front-end Developers	Medium (JS/CSS, APIs)	Branding, lightweight integrations
Objects	Low-code database entities with built-in forms and REST APIs	Custom data models (e.g., Invoices, Tickets, Contacts)	Admins, Analysts	Low (UI-driven, optional Groovy)	Business users, data owners

Liferay 7.4 Components (2 of 3)

Component	Purpose	Primary Use Cases	Customizable By	Tech Skills Needed	Best For
App Builder	Builds web applications on top of Objects	Form-based apps, ticketing, workflows, internal tools	Admins, Power Users	Low (drag-and-drop)	Citizen developers
Web Content (CMS)	Manage and publish static or structured content	News, FAQs, announcements, help articles	Content Authors	Low	Editors, marketers
Style Books	Define branding: fonts, colors, spacing	Brand identity, quick styling without theme development	Designers, Admins	Low	Branding teams
Themes	Full layout and visual customization across portal	Branded sites, deep UI customization	Front-end Devs	High (Freemarker, CSS, JS)	Designers, UI developers

Liferay 7.4 Components (3 of 3)

Component	Purpose	Primary Use Cases	Customizable By	Tech Skills Needed	Best For
Workflows	Define approval processes and task sequences	Content approval, form submission routing	Admins, Workflow Designers	Medium (logic-based setup)	Governance, compliance teams
Job Scheduler	Run background tasks at scheduled times	Data syncs, cleanup scripts, reporting jobs	Developers	High (Java)	Admins, backend devs
JSON Web Services	Exposes Liferay content/data as REST or GraphQL APIs	Mobile apps, system integrations, headless front-ends	Developers	Medium-High (API, JSON, OAuth)	Integration teams

Feature Cheat Sheet (1 of 4)

If You Want To...	Use This	Role
Build forms/apps without code	App Builder + Objects	Marketer
Add branding without dev effort	Style Book	Marketer
Personalize content and experiences	Audience Targeting/Segments	Marketer
Store and manage structured content	Web Content (Structured)	Content Manager
Manage documents, images, blogs, forums	Documents & Media, Blogs, etc.	Content Manager
Support multiple languages	Localization/Translation	Content Manager
Organize content with categories/tags	Asset Categorization/Tagging	Content Manager

Feature Cheat Sheet (2 of 4)

If You Want To...	Use This	Role
Build a marketing landing page	Fragments + Style Book	Front End Developer
Automate publishing approval	Workflows	Front End Developer
Inject custom JS/CSS into pages	Client Extension	Front End Developer
Ensure mobile/responsive design	Fragments, Themes	Front End Developer
Add business logic or data-driven UI	Portlet or Client Extension	Back End Developer
Send scheduled reminders or syncs	Job Scheduler	Back End Developer
Connect to mobile app or another platform	JSON Web Services	Back End Developer

Feature Cheat Sheet (3 of 4)

If You Want To...	Use This	Role
Replace spreadsheets with custom records	Objects	Back End Developer
Integrate with external systems	Integration (LDAP, SSO, APIs)	Back End Developer
Manage users, roles, and permissions	Users & Roles	Administrator
Create and organize sites/pages	Site Builder	Administrator
Configure search and indexing	Search	Administrator
Set up notifications and subscriptions	Notifications/Subscriptions	Administrator
Track analytics and reporting	Analytics/Reporting	Administrator

Feature Cheat Sheet (4 of 4)

If You Want To...	Use This	Role
Manage staging and publishing	Staging/Publishing	Administrator
Upgrade and maintain Liferay	Upgrades/Maintenance	Administrator

Workshop - Install Dev Workspace

- Time to install everything on your local machines.
- Clone the github repo here:
<https://github.com/cwilde1/liferay-training.git>
- Follow steps in README.md

Admin & Site Basics – Hands-On

- Create users and roles
- Set permissions
- Build basic site and pages

Agenda - Day 3

- Objects & Workflows (Demo)
- Headless Apis via JSON (Demo/Workshop)
- Job Scheduler (Demo/Workshop)
- Q&A
- Liferay Monitoring (Demo)
- Stepping through code with debugger
- Course Reviews

Liferay Monitoring (ELK Stack)

ELK = Elasticsearch, Logstash, Kibana (free & open source)

- Centralized log aggregation & analysis for Liferay logs)
- Collects logs via Logstash or Beats, stores in Elasticsearch
- Kibana dashboards enable powerful search, filtering & visualization
- Great for troubleshooting errors, auditing, and monitoring multi-server environments
- Setup requires managing 3 services; resource-heavy but highly scalable

Liferay Monitoring (Glowroot)

- Lightweight Java agent for real-time APM & performance profiling
- Monitors JVM metrics: response times, SQL, errors, heap, GC, thread dumps
- Intuitive dashboard for identifying slow transactions and memory issues
- Easy setup, minimal overhead, ideal for deep code-level insights
- Focused on single JVM monitoring, not centralized log management
- Configure by adding:
`-javaagent:/path/to/glowroot/glowroot.jar` to Tomcat's JVM startup options.

Liferay Monitoring (Tomcat Manager)

Tomcat Manager is a web-based administration tool for deploying, monitoring, and managing applications on an Apache Tomcat server.

- Should not be used with Liferay, due to potential security risks and deployment conflicts.

Get CompanyId & GroupId via Groovy Script

```
import com.liferay.portal.kernel.service.*
```

```
CompanyLocalServiceUtil.getCompanies().each { company ->
```

```
    println "CompanyId: ${company.companyId}"
```

```
    GroupLocalServiceUtil.getGroups(company.companyId, 0, true).each {  
group ->
```

```
    println " ${group.getNameCurrentValue()} -> groupId: ${group.groupId}"
```

```
}
```

```
}
```

Connect Service Builder to Existing DB Table

- In service.xml, set the <entity table="YourTableName"> attribute to match your DB table.
- Ensure entity fields match your table's columns (names/types).
- Run Service Builder to generate service/persistence classes.

Key Considerations for existing table

- Table must have a primary key.
- Schema must match Service Builder expectations.
- Backup your DB before connecting.
- Manual tweaks may be needed for custom constraints or naming.